

Documentación del Proyecto: Gym Tracker

Autor: Óscar Marqués Juan

Dni: 78221887Z

Repositorio del código en línea: github.com/oscarMJ04

1. Introducción.....	2
2. Tecnologías y dependencias.....	2
3. Arquitectura y Tecnologías	3
4. Modelo de Datos y Entidades.....	3
5. Manual de Usuario	4
5.1 Menú Principal.....	4
5.2 Registro y Autenticación	4
5.3. Creación de Rutinas y Entrenamiento Activo.....	5
5.4 Catálogo y Ejercicios Personalizados	6
5.5 Muro Social (Feed) e Interacciones	7
5.5. Buscador y Conexiones (Red Social)	8
5.6 Historial personal	9
6. Integridad de datos y estrategias de borrado	10
6.1 Borrado de ejercicios desde el catálogo	11
6.2 Borrado de ejercicios desde la rutina	11
6.2.1 Desde edición	11
6.2.2 Desde el entrenamiento en marcha.....	11
6.3 Borrado de rutinas	11
6.4 Eliminación de sesiones	11
7. Instalación y Ejecución	11
8. Conclusiones	12

1. Introducción

Gym Tracker es una aplicación web integral diseñada para la gestión, seguimiento y socialización de rutinas de entrenamiento en el gimnasio. A diferencia de un simple bloc de notas, la plataforma adopta un enfoque social (similar a redes como Hevy o Strava), permitiendo a los usuarios no solo registrar sus marcas personales, sino también interactuar con otros atletas mediante un muro comunitario, sistema de seguimiento, comentarios y reacciones.

2. Tecnologías y dependencias

Componente	Tecnología
Lenguaje backend	Python
Framework web	Flask
Gestión de sesiones	Flask-Login
Hash de contraseñas	Werkzeug Security
Base de datos	Redis
ORM / capa de persistencia	Sirope
Motor de plantillas	Jinja2 (incluido en Flask)
Frontend	HTML5, CSS3, JavaScript vanilla

Sirope actúa como capa de abstracción sobre Redis, permitiendo serializar y deserializar objetos Python directamente en la base de datos mediante un sistema de OIDs (Object Identifiers). Esto elimina la necesidad de mapear manualmente los atributos de cada clase a estructuras de clave-valor.

Flask-Login gestiona el ciclo de vida de la sesión de usuario, protegiendo las rutas que requieren autenticación mediante el decorador `@login_required`.

3. Arquitectura y Tecnologías

El proyecto se ha desarrollado bajo una arquitectura cliente-servidor robusta:

- **Backend:** Desarrollado en **Python 3** utilizando el micro-framework **Flask**.
- **Base de Datos NoSQL:** Se ha empleado **REDIS** a través de la capa de abstracción **Sirope**, permitiendo un almacenamiento rápido y flexible basado en objetos (OIDs).

- **Frontend:** Construido con **HTML5**, **Jinja2** para la renderización de plantillas del lado del servidor, y **Pico CSS** para garantizar un diseño responsivo, limpio y accesible sin sobrecargar el cliente con frameworks pesados.
- **Interactividad:** Uso de **JavaScript vanilla (AJAX/Fetch API)** para operaciones asíncronas, como la creación de ejercicios en tiempo real durante un entrenamiento sin recargar la página.

Para garantizar el **Diseño Modular**, la lógica de negocio se ha desacoplado utilizando *Blueprints* de Flask en cuatro módulos principales:

1. `auth.py`: Autenticación de usuarios y motor de relaciones sociales (solicitudes y seguimientos).
2. `modulo_ejercicios.py`: Gestión del catálogo global y los ejercicios personalizados.
3. `modulo_rutinas.py`: Lógica de creación (CRUD) de planes de entrenamiento y la ejecución activa de las sesiones.
4. `modulo_registros.py`: Gestión del historial individual y renderizado del muro social (Feed).

4. Modelo de Datos y Entidades

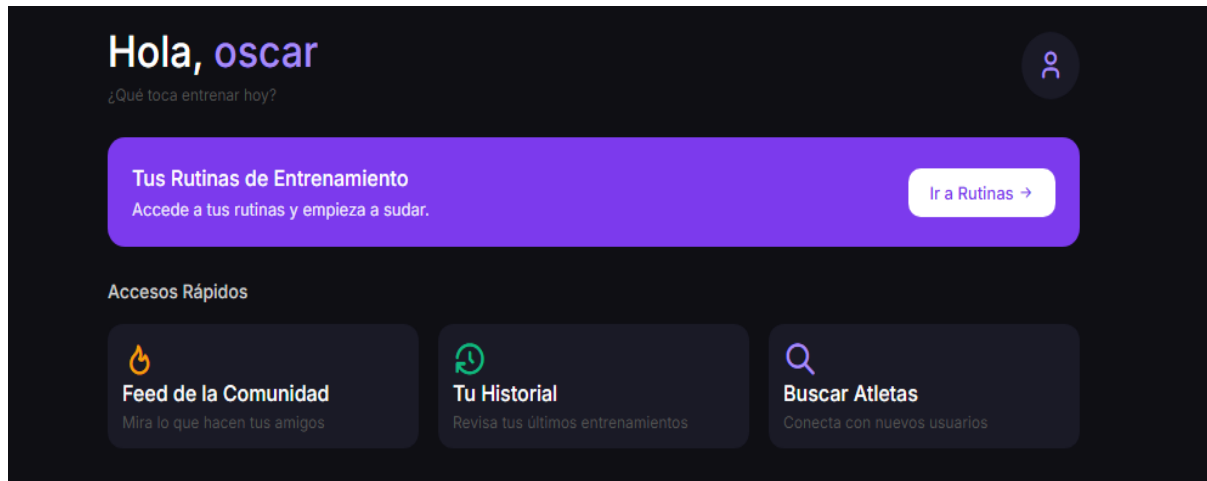
El sistema gestiona la complejidad a través de cuatro entidades principales que se relacionan dinámicamente:

- **Usuario (User):** Gestiona la autenticación mediante hashes de contraseña (Werkzeug). Incluye atributos complejos como `siguiendo` (lista de usuarios conectados) y `solicitudes_pendientes` para gestionar la red social.
- **Ejercicio (Ejercicio):** Distingue entre ejercicios base del sistema (`es_defecto=True`, inmutables) y ejercicios creados por los usuarios (`user_id`).
- **Rutina (Rutina):** Actúa como un contenedor que agrupa una serie de referencias (OIDs) a ejercicios del catálogo para un usuario específico.
- **Sesión de Entrenamiento (SesionEntrenamiento):** Es la entidad más compleja. Almacena una instantánea inmutable del entrenamiento completado. Guarda el nombre de usuario creador, la fecha, duración, y una estructura anidada de diccionarios con las series, repeticiones y pesos. Además, incluye colecciones para fuegos (likes) y comentarios.

5. Manual de Usuario

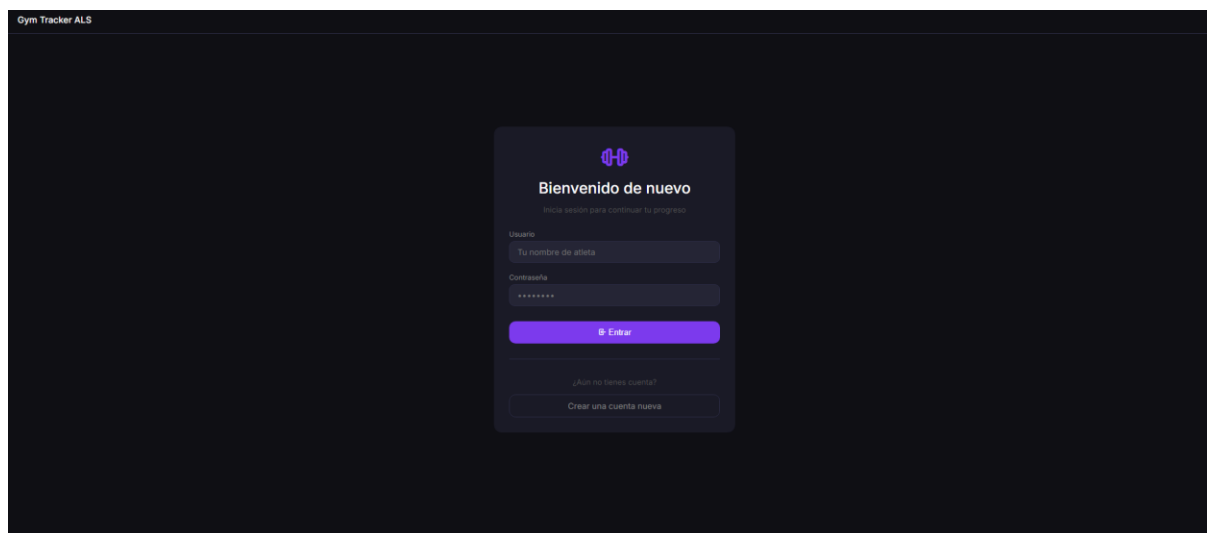
5.1 Menú Principal

Esta es la pantalla que el usuario ve nada más ingresar en la aplicación. En esta pantalla verá las funcionalidades que puede realizar: acceder a rutinas, ir al feed de la comunidad, ver su historial y buscar a otros atletas.



5.2 Registro y Autenticación

Al acceder a la aplicación, el usuario es recibido por el panel de inicio de sesión. Si es nuevo, puede crear una cuenta de forma segura.

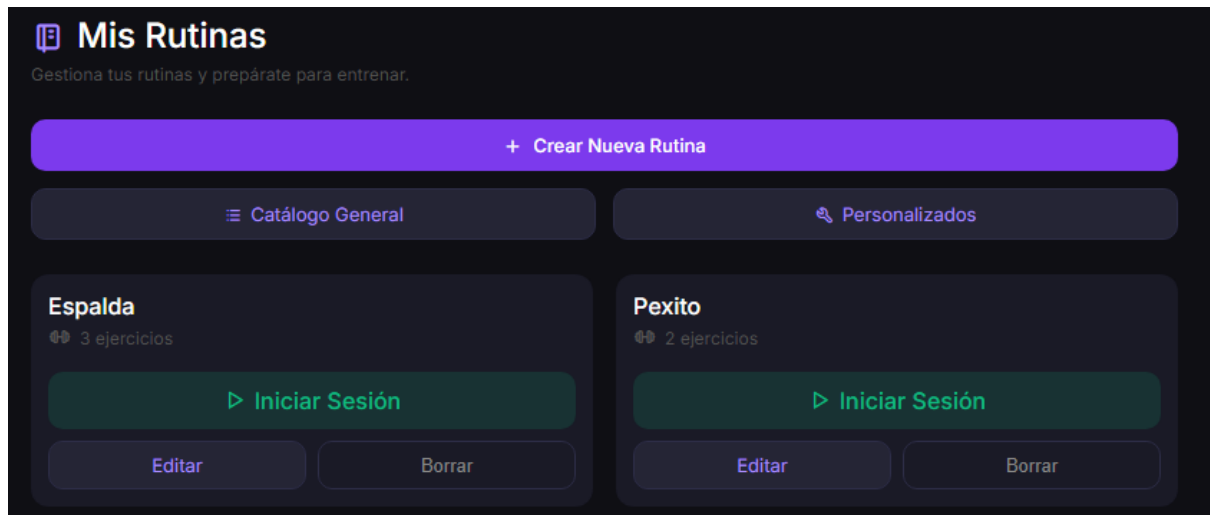


5.3. Creación de Rutinas y Entrenamiento Activo

En la pestaña "Rutinas", el usuario puede diseñar planes seleccionando ejercicios del catálogo. Al pulsar "Entrenar", se despliega una interfaz de entrenamiento activo que incluye:

- Un cronómetro en tiempo real.

- Filas dinámicas para añadir series (Peso y Repeticiones).
- La posibilidad de inyectar nuevos ejercicios sobre la marcha mediante un modal asíncrono.



Espalda

Entrenamiento en curso

00:03

×

Jalon al pecho

S1

Kg

Reps

🗑

+ Añadir Serie

Dominadas

S1

Kg

Reps

🗑

+ Añadir Serie

Pull over

S1

Kg

Reps

🗑

+ Añadir Serie

¿Añadir ejercicio sobre la marcha?

-- Catálogo --

▼

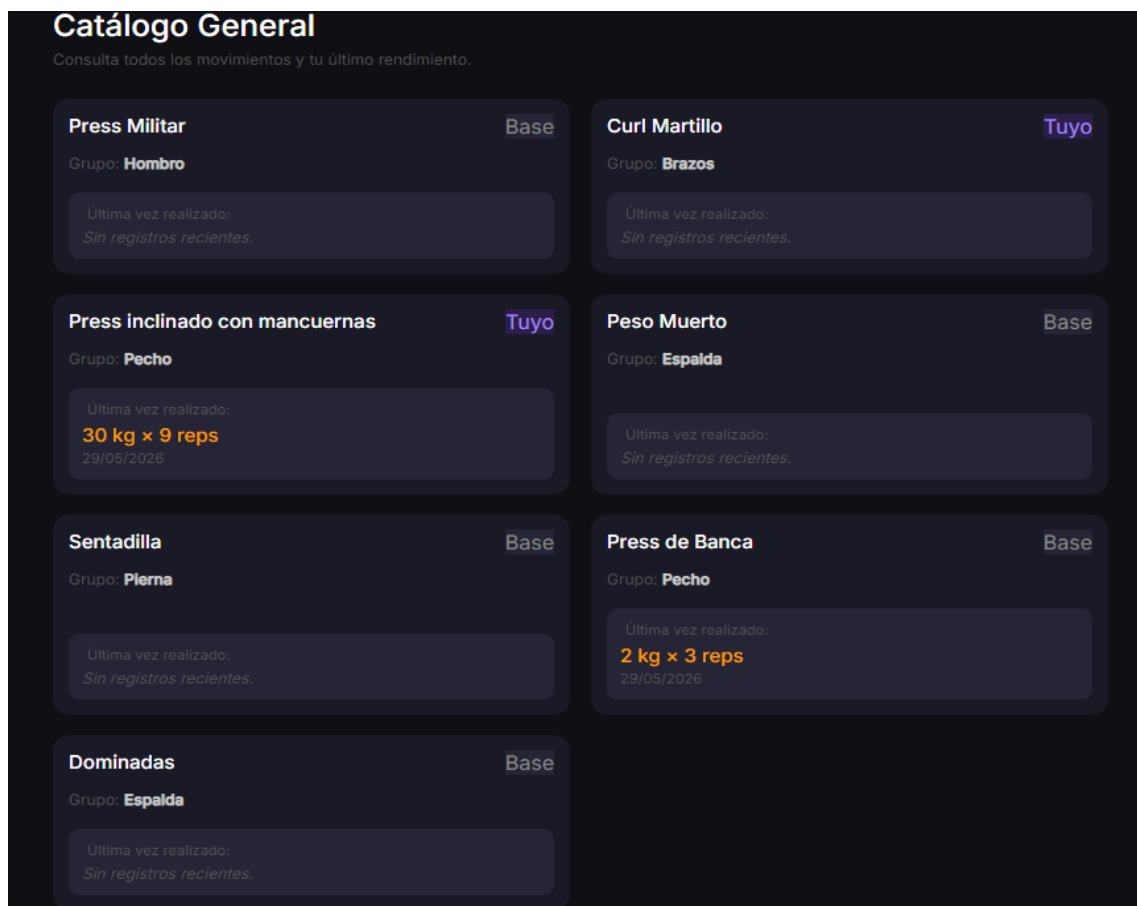
Añadir

🔗 Crear ejercicio nuevo

✓ Finalizar y Publicar

5.4 Catálogo y Ejercicios Personalizados

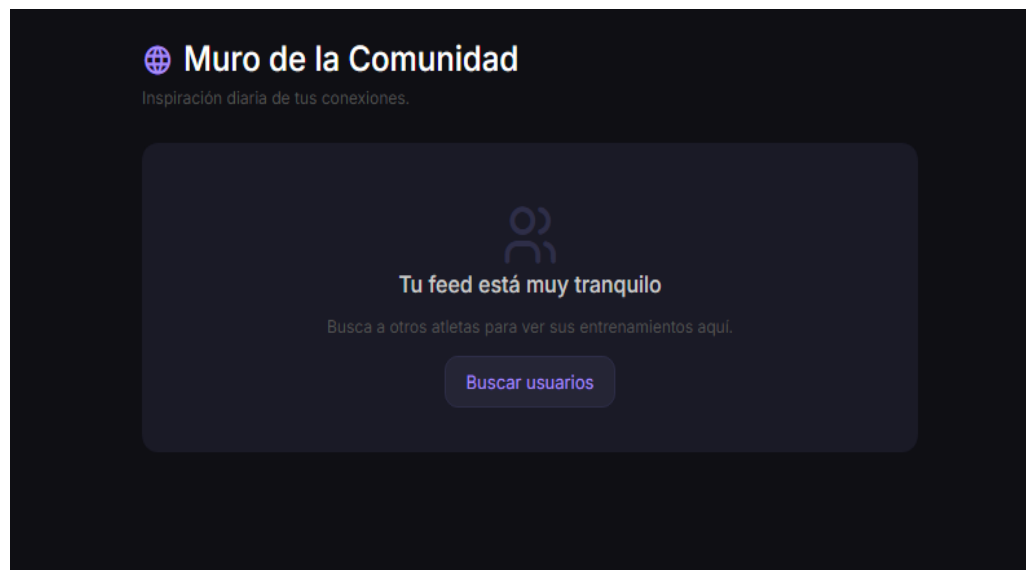
Desde el menú de rutinas de entrenamiento, el usuario puede acceder al "Catálogo General". Aquí visualizará los ejercicios del sistema junto con su **última marca registrada** (calculada dinámicamente a partir de su historial). El usuario puede crear sus propios movimientos desde "Ejercicios Personalizados", los cuales se añadirán a su catálogo privado.



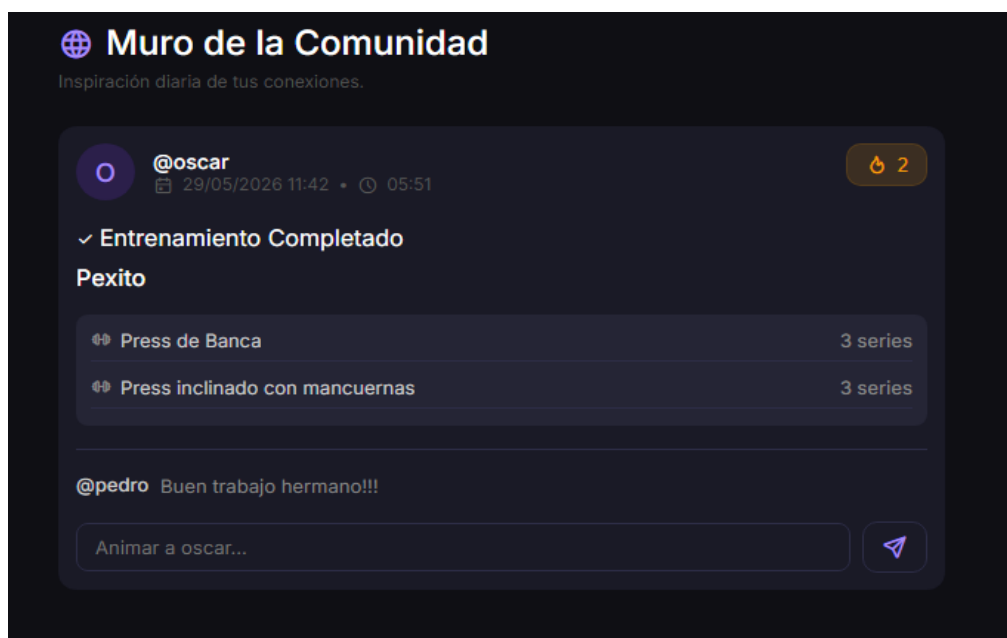
5.5 Muro Social (Feed) e Interacciones

Al finalizar un entrenamiento, este se publica automáticamente. En la pestaña "Feed / Muro", el usuario puede ver sus sesiones y las de los atletas con los que ha conectado.

En este muro, es posible interactuar dejando comentarios o pulsando el botón de



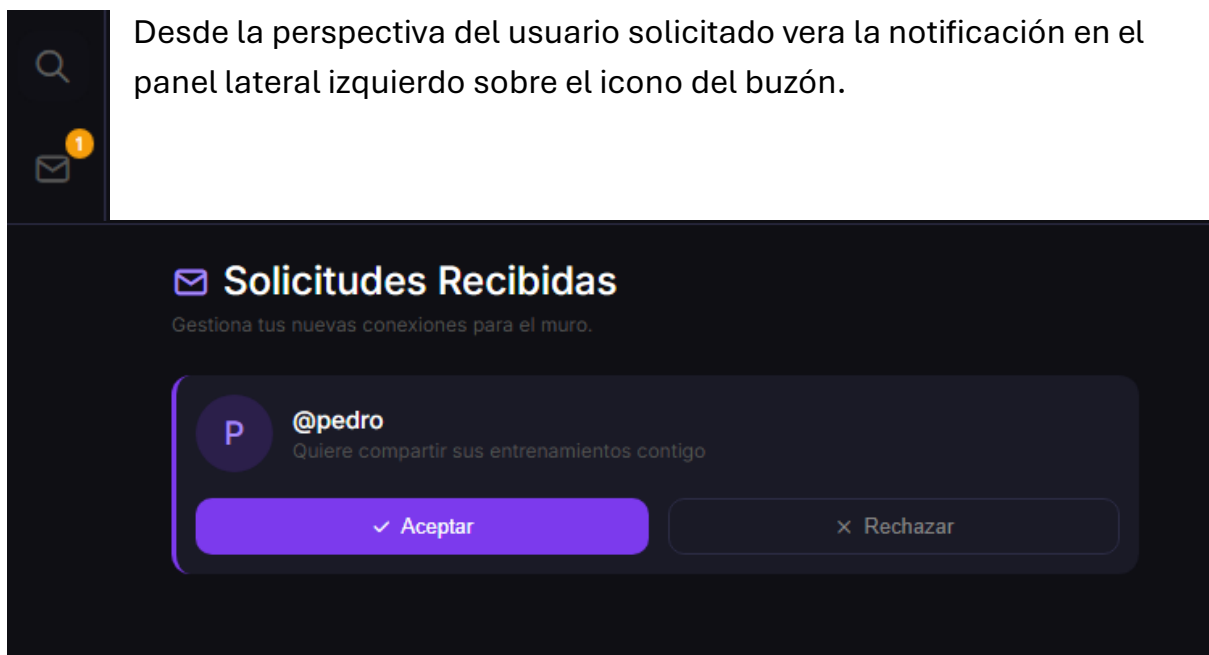
"Fuego" (🔥).



5.5. Buscador y Conexiones (Red Social)

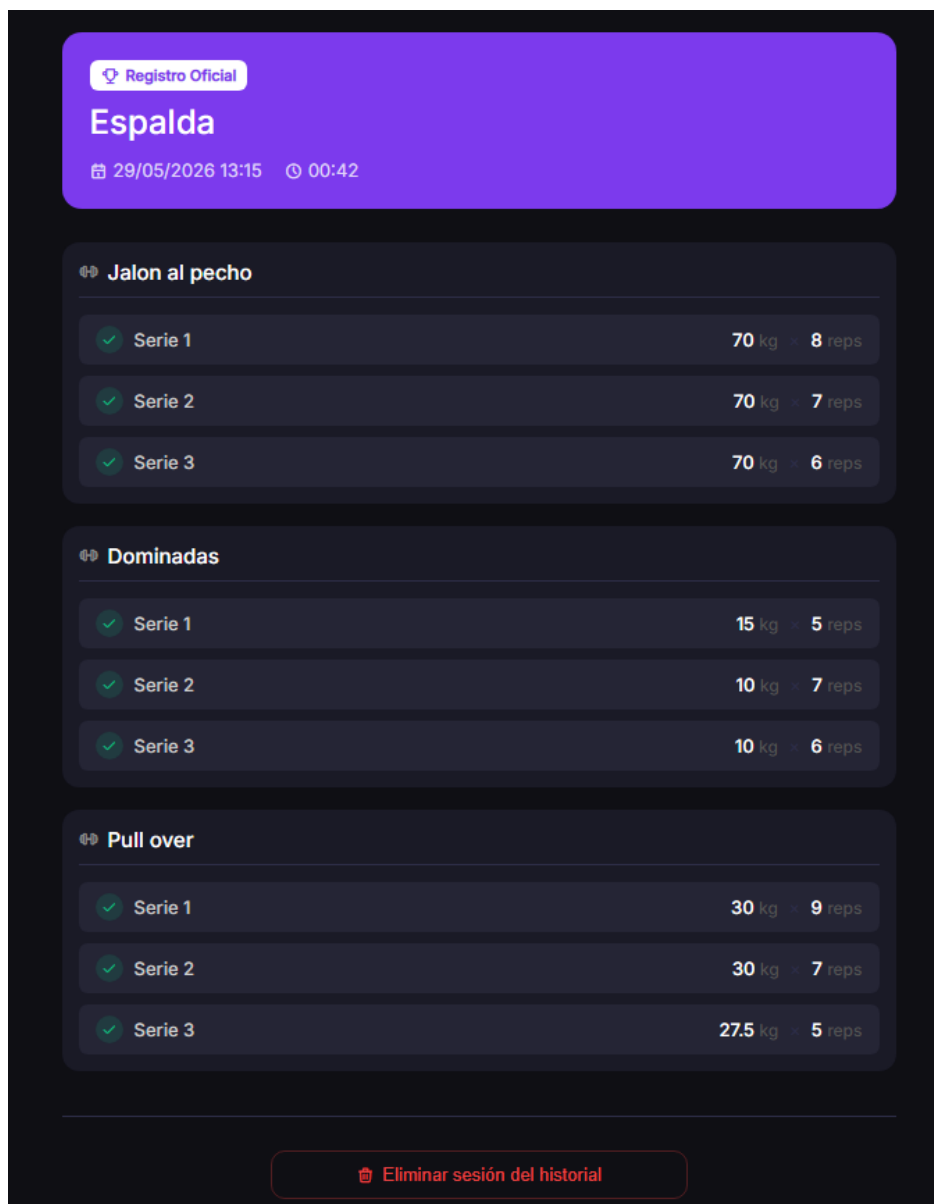
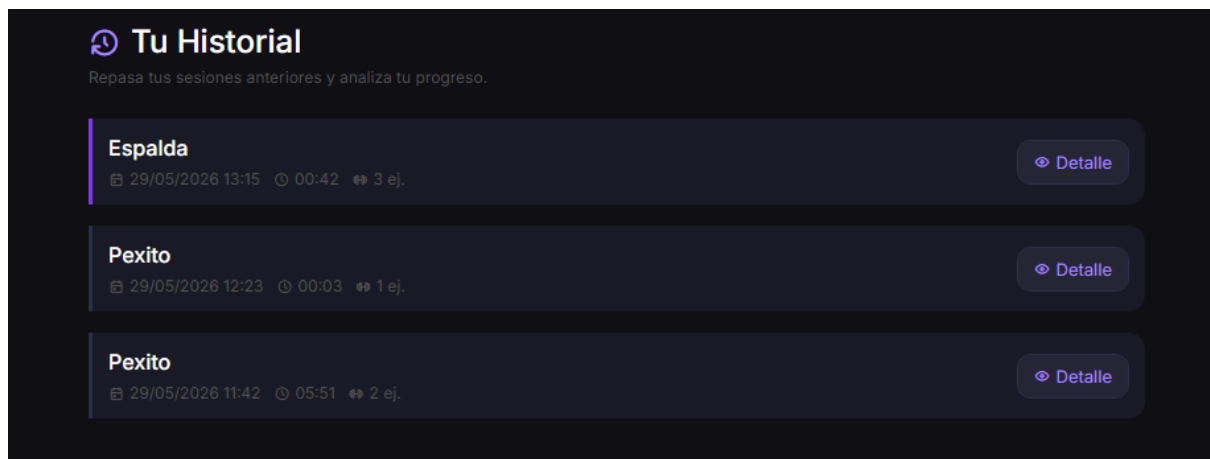
Desde el menú principal o desde la barra lateral (Símbolo de lupa) el usuario puede acceder al sistema de búsqueda de usuarios y enviar solicitudes de conexión.

1. El receptor verá una notificación en su buzón de la barra lateral.
2. Al aceptar, los muros de ambos usuarios quedan sincronizados.



5.6 Historial personal

En este apartado se verá una recopilación de entrenamientos realizados, con la opción de verlos en detalle.



También se nos permite, una vez en los detalles del entrenamiento, borrar el historial de esa sesión, lo cual borrará la sesión tanto del historial como del feed.

6. Integridad de datos y estrategias de borrado

Uno de los aspectos más relevantes del proyecto es el tratamiento diferenciado de los borrados, diseñado para preservar la coherencia histórica de los datos.

6.1 Borrado de ejercicios desde el catálogo

Cuando un ejercicio del catálogo ya no es relevante para el usuario, puede borrarse desde el catálogo personalizado. No se pueden borrar los ejercicios base de la app. De esta manera también se borra el mismo ejercicio de cualquier rutina en la que estuviera referenciado.

6.2 Borrado de ejercicios desde la rutina

6.2.1 Desde edición

El borrado desde esta pantalla borra únicamente el propio ejercicio de la rutina misma. Este borrado es persistente y afecta a usos futuros de la propia rutina.

6.2.2 Desde el entrenamiento en marcha

Una vez iniciado el entrenamiento la aplicación permite varias funciones, entre ellas borrar un ejercicio activo. Este tipo de borrado solo afecta a la sesión actual, es decir, en el feed y en el registro personal aparecerá dicho borrado, pero la rutina, una vez terminado el entrenamiento, volverá al estado anterior.

6.3 Borrado de rutinas

La eliminación de una rutina es una operación ligera: solo destruye el objeto Rutina en Redis. Sin embargo, los entrenamientos realizados con esa rutina se mantienen en el historial personal, ya que la sesión almacena el nombre como texto plano en el momento de su creación y no depende de una referencia por OID.

6.4 Eliminación de sesiones

El usuario puede eliminar sesiones individuales de su historial. La operación solo permite borrar sesiones propias. Este borrado es más fuerte que el anterior, ya que este también borra el historial de ese mismo registro del feed comunitario.

7. Instalación y Ejecución

Requisitos Previos

- Python 3.12+
- Servidor Redis en ejecución (local o WSL).

Configuración del Entorno

- Crear entorno: `python -m venv venv`
- Activar (Windows): `.\venv\Scripts\activate`
- Activar (Linux/WSL): `source venv/bin/activate`

Dependencias y Arranque

- Instalar librerías: `pip install flask flask-login werkzeug sirope redis`
- Ejecutar: Navegar a la carpeta `src/` y lanzar el comando `flask run`.

La aplicación queda disponible en `http://127.0.0.1:5000`. En el primer arranque, el sistema inicializa automáticamente el catálogo base de ejercicios si Redis está vacío.

8. Conclusiones

El desarrollo de esta aplicación ha constituido un proceso de aprendizaje integral sobre el ciclo de vida completo de un producto software. Se ha logrado con éxito la transición desde una idea abstracta hasta una plataforma funcional, robusta y con una experiencia de usuario cuidada.

Como principales conclusiones del proyecto, destacamos:

1. Valor de la UX: Hemos comprobado que el diseño de interfaz no es un elemento estético, sino funcional. La implementación de modales de confirmación y salvaguardas de navegación no solo mejoró la estética, sino que incrementó drásticamente la fiabilidad percibida por el usuario.
2. Robustez de la arquitectura: El uso de un *stack* tecnológico basado en Flask y Sirope ha demostrado ser una solución excepcionalmente ágil, permitiendo realizar cambios significativos en el modelo de datos y la interfaz sin necesidad de realizar una reescritura masiva del código.
3. Competencia Full-Stack: Este proyecto ha permitido unificar conocimientos de back-end (gestión de rutas, persistencia) con desafíos complejos de front-end (manejo de DOM, eventos, persistencia en sesión), consolidando una visión holística del desarrollo de aplicaciones web.